

Assignment-Module-45

Map and Generics

Q.1Ans-A Map is a collection in Java that stores data as key-value pairs, where each key is unique.

Q.2 Ans- Ans: The commonly used implementations of Map in Java are HashMap, TreeMap, LinkedHashMap, and ConcurrentHashMap.

Q.3Ans: HashMap is an unordered collection that uses hashing to store the key-value pairs, while TreeMap is a sorted collection that stores the key-value pairs in a sorted order based on the natural order of the keys or a custom Comparator.

Q.4 Ans. we can check if a key exists in a Map in Java using the `containsKey()` method or the `get()` method. The `containsKey()` method returns a boolean value indicating whether the Map contains the specified key, while the `get()` method returns the value associated with the specified key, or null if the key is not present in the Map.

Q.5Ans:- Generics in Java are used to provide type safety and reduce code redundancy by allowing the use of generic types. It allows classes, methods, and interfaces to be written generically, without specifying the type of data being used.

Q.6Ans: The benefits of using Generics in Java are:

Type safety

Code reusability

Improved readability

Reduced code redundancy

Improved performance

Q.7Ans: A Generic Class in Java is a class that can work with different types of data. It is defined using a type parameter enclosed in angle brackets, which represents the type of data being used.

Q.8Ans: A Type Parameter in Java Generics is a placeholder for the type of data that is used by a generic class or method. It is defined using a single uppercase letter enclosed in angle brackets, such as <T> or <E>.

Q9 Ans. A Generic Method in Java is a method that can work with different types of data. It is defined using a type parameter enclosed in angle brackets, which represents the type of data being used.

Q.10Ans-: ArrayList is a non-generic class, while ArrayList<T> is a generic class. ArrayList<T> provides type safety, as it can only store elements of the specified type, whereas ArrayList can store any type of element.

The primary difference between **ArrayList** and **ArrayList<T>** in Java lies in the use of **generics**, which provides type safety and eliminates the need for explicit type casting. Here's a detailed explanation:

Feature	ArrayList	ArrayList<T>
---------	-----------	--------------

Type Specification	Does not specify a type for the elements; it is a raw type.	Specifies a generic type (T) for the elements.
--------------------	--	---

Type Safety	Not type-safe; elements can be of any type.	Type-safe; ensures that only elements of type T are added.
-------------	---	---

Explicit Casting	Requires explicit type casting when retrieving elements.	No need for explicit casting as the type is known.
Compile-Time Checks	Compile-time type checks are not enforced , leading to potential runtime errors.	Enforces compile-time type checks , reducing the chances of runtime errors.
Recommended Use	Used in legacy code or where type safety is not critical.	Preferred in modern Java applications to ensure type safety.